

1. [수업자료] 컴퓨터의 이해

컴퓨터와 소통하는 방법, 0과 1

컴퓨터는 **기계어**라고 불리는 **0과 1의 이진 코드**를 통해 작동합니다. 이는 컴퓨터의 가장 기본적인 언어로, 모든 디지털 장치(TV, 컴퓨터, 에어컨 등)는 켜짐(1)과 꺼짐(0)의 상태를 통해 기능을 수행합니다. 복잡한 기계어를 사람이 쉽게 활용할 수 있도록 돕는 것이 **엔지니어의 역할**입니다.

쉬운설명

기계어란 무엇일까요? 컴퓨터는 우리가 쓰는 언어를 이해하지 못해요. 대신 0과 1이라는 두 가지 숫자만으로 모든 것을 처리하는데, 불이 켜진 상태를 1, 꺼진 상태를 0으로 비유할 수 있죠.

우리 일상생활 속 기계어 예시:

- TV 켜고 끄기: TV를 켜면 1, 끄면 0이라고 생각할 수 있어요.
- 에어컨 On/Off: 에어컨도 마찬가지로 켜면 1, 끄면 0이죠. 이처럼 모든 디지털 기기는 이 0과 1의 조합을 통해 작동하고 있어요.

왜 기계어를 알아야 할까요? 우리가 직접 0과 1로 컴퓨터에게 명령을 내리는 것은 너무 복잡하고 어려워요. 예를 들어, 컴퓨터에게 "안녕하세요"라고 말하려면 엄청나게 긴 0과 1의 조합을 입력해야 할 거예요. 그래서 **엔지니어(개발자)**들이 이 복잡한 기계어를 사람이 쉽게 쓸 수 있는 **프로그래밍 언어**로 바꿔주는 도구를 만들어서 우리가 컴퓨터를 편리하게 사용할 수 있게 해주는 것이죠. 덕분에 우리는 굳이 0과 1을 외울 필요 없이 편하게 컴퓨터와 소통할 수 있게 된답니다!

프로그래밍 언어로 소통하는 개발자

개발자는 **프로그래밍 언어**를 사용하여 컴퓨터와 소통합니다. 자바스크립트, 스위프트, 코틀린, C++, Go, Rust, C, 어셈블리, 자바, C#, 파이썬 등 다양한 프로그래밍 언어가 존재하며, 이 언어들은 컴퓨터 위에서 실행됩니다. 프로그래밍 언어로 작성된 코드는 **인터프리터**나 **컴파일러**를 통해 기계어로 번역된 후 컴퓨터에 의해 인식됩니다.

쉬운설명

프로그래밍 언어는 왜 필요할까요? 앞서 기계어가 0과 1로 이루어진 컴퓨터의 언어라고 했죠? 하지만 사람이 그 복잡한 0과 1을 직접 입력하기는 불가능에 가까워요. 그래서 개발자들은 사람의 언어와 더 비슷하면서도 컴퓨터가 이해할 수 있는 중간 언어를 만들었어요. 이것이 바로 다양한 프로그래밍 언어입니다.

세상에는 정말 많은 프로그래밍 언어가 있어요. 파이썬(Python), 자바(Java), C++, 자바스크립트(JavaScript) 등이 대표적이죠. 이 언어들은 각각의 목적과 쓰임새가 달라서, 마치 우리가 영어, 일본어, 중국어를 배우듯 개발자들은 필요한 언어를 배워서 컴퓨터에게 명령을 내린답니다.

프로그래밍 언어가 기계어로 바뀌는 과정:

1. **코드 작성:** 개발자가 파이썬이나 자바 같은 프로그래밍 언어로 코드를 작성해요 (이 코드를 소스 코드라고도 해요).
2. **번역:** 이 코드는 컴퓨터가 이해할 수 있는 기계어(0과 1)로 번역되어야 해요. 이때 **인터프리터(interpreter)**나 **컴파일러(compiler)**라는 특별한 번역 프로그램이 사용됩니다.
 - **컴파일러:** 코드를 한꺼번에 기계어로 번역해서 실행 파일(**exe** 파일 같은)을 만들어요. 번역은 한 번만 하면 되기 때문에 실행 속도가 빨라요.

- **인터프리터:** 코드를 한 줄씩 읽어서 바로바로 기계어로 번역하고 실행해요. 번역과 실행이 동시에 이루어져 개발 과정에서 편리하지만, 실행 속도는 컴파일러 방식보다 느릴 수 있어요.

3. 컴퓨터 인식: 번역된 기계어를 컴퓨터가 이해하고 명령을 실행합니다.

결국 프로그래밍 언어는 개발자가 컴퓨터에게 "이렇게 해줘!"라고 쉽게 말하기 위한 도구이고, 인터프리터나 컴파일러는 그 말을 컴퓨터가 알아듣도록 통역해주는 역할을 하는 것이죠!

Low level vs High level language

프로그래밍 언어는 **Low level language**와 **High level language**로 나눌 수 있습니다.

Low level language는 기계어와 직접적으로 더 가까워 컴퓨터를 더 직접적으로 제어할 수 있고 실행 속도가 빠릅니다. 하지만 **작성이 어렵고 오류 발생 가능성이 높습니다**. 반면 **High level language**는 이러한 단점을 보완하여 사람이 작성하기 쉽고 배우기 편리하게 고안된 언어입니다. High level이라고 해서 무조건 더 상위의 언어라는 의미는 아닙니다.

쉬운설명

Low level 언어와 High level 언어의 차이점: 프로그래밍 언어는 컴퓨터와 얼마나 가깝게 대화하느냐에 따라 크게 두 가지 종류로 나눌 수 있어요.

Low level language (저급 언어)

컴퓨터와 가까운 언어: 기계어(0과 1)와 가장 가까운 형태의 언어예요. '어셈블리(Assembly)' 같은 언어가 대표적이죠.

- **장점:** 컴퓨터의 하드웨어(CPU, 메모리 등)를 아주 세밀하게 직접 제어할 수 있어서, 프로그램을 매우 빠르고 효율적으로 만들 수 있어요. 마치 자동차의 엔진 부품 하나하나를 직접 조작하는 것과 같아요.
- **단점:** 사람이 이해하고 작성하기가 너무 어려워요. 코드가 매우 복잡하고, 작은 실수 하나에도 프로그램이 오작동하기 쉬워요. 초보자가 다루기엔 거의 불가능에 가깝습니다.

High level language (고급 언어)

사람과 가까운 언어: 우리가 사용하는 자연어(영어)와 비슷한 형태로 만들어진 언어예요. 파이썬, 자바, C++ 등이 여기에 해당하죠.

- **장점:** 배우기 쉽고, 코드를 작성하기가 편리해요. 복잡한 명령어를 몇 줄의 간단한 코드로 표현할 수 있어서 개발 속도가 빠르고, 오류를 줄이기도 쉬워요. 마치 자동차를 운전할 때 핸들, 가속페달, 브레이크만 조작하는 것과 같아요. 내부 복잡한 작동 방식은 몰라도 운전은 할 수 있죠!
- **단점:** Low level 언어만큼 컴퓨터 하드웨어를 직접적으로 제어하기는 어려워요. 번역 과정이 필요하기 때문에 Low level 언어보다는 실행 속도가 약간 느릴 수 있습니다.

핵심: High level 언어가 "더 고차원적"이라는 의미보다는, "**사람이 사용하기 더 편리하게 만들어졌다**" 는 의미로 이해하는 것이 좋아요. 개발 목적과 성능 요구사항에 따라 적절한 언어를 선택하게 됩니다.

클라이언트와 서버 (Front-end와 Back-end)

웹 서비스는 크게 **클라이언트(Front-end)** 와 **서버(Back-end)** 로 구성됩니다. 클라이언트는 사용자가 직접 마주하는 부분(예: 배달앱 UI)으로, 웹 개발에서는 '프론트엔드'라고 부릅니다. 서버는 클라이언트의 요청을 처리하고 데이터를 저장하며, 웹 개발에서는 '백엔드'라고 부릅니다. 예를 들어, 배달 앱에서 음식을 주문하면(클라이언트), 이 주문 정보는 서버로 전달되고(배달음식 주문), 서버는 주문을 처리한 후 완료/배달 이동 정보를 클라이언트로 다시 보냅니다.

🔍 쉬운설명

클라이언트와 서버, 그리고 프론트엔드/백엔드: 우리가 웹사이트나 앱을 사용할 때, 눈에 보이는 부분과 눈에 보이지 않는 처리 부분이 나눠져 있어요.

클라이언트 (Client)

사용자 인터페이스: 사용자가 직접 보고 상호작용하는 모든 부분이에요. 예시: 스마트폰의 배달 앱 화면, 웹사이트의 디자인, 버튼, 이미지 등. 사용자가 정보를 입력하거나 버튼을 누르는 등의 모든 행동이 클라이언트에서 이루어져요. 웹에서는 클라이언트 부분을 **프론트엔드(Front-end)** 라고 부릅니다.

서버 (Server)

정보 처리 및 저장: 클라이언트의 요청을 받아서 처리하고, 데이터를 저장하거나 관리하는 부분이에요. 사용자에게는 직접 보이지 않지만, **서비스의 핵심 기능을 담당해요**. 예시: 배달 앱에서 "치킨 주문" 버튼을 누르면, 이 주문 정보가 서버로 전송되어 "어떤 치킨을 누가 어디로 주문했는지"를 기록하고 처리해요. 웹에서는 서버 부분을 **백엔드(Back-end)** 라고 부릅니다.

배달 앱 예시로 이해하기:

- 당신 (사용자): 배달 앱을 열고 원하는 치킨을 선택한 후 "주문하기" 버튼을 눌러요. → **클라이언트(프론트엔드)** 에서 일어나는 일.
- 배달 앱 서버: 당신이 누른 주문 정보를 받아서, "누가, 어떤 메뉴를, 어디로 주문했다"는 정보를 데이터베이스에 저장하고, 결제 처리를 하고, 가게에 주문을 전달해요. → **서버(백엔드)** 에서 일어나는 일.
- 다시 당신에게: 서버에서 "주문 완료! 배달 예상 시간 30분"이라는 메시지를 당신의 앱 화면에 띄워줘요. → **클라이언트(프론트엔드)** 에서 다시 보여주는 일.

이처럼 클라이언트와 서버는 서로 데이터를 주고받으며 하나의 서비스를 완성하는 중요한 관계랍니다!

컨트롤이 다른 이유, 운영체제

운영체제(Operating System, OS) 는 컴퓨팅 기계를 쉽게 다루기 위한 **시스템 소프트웨어**입니다. MacOS, Windows, Linux, Android, iOS 등 다양한 운영체제가 있으며, 각 운영체제는 하드웨어(카메라 등)에 접근하고 제어하는 코드를 다르게 제공합니다. 이로 인해 **개발자는 운영체제별로 애플리케이션을 다르게 제작해야 합니다**. 운영체제는 사용자가 하드웨어 접근 권한 등을 쉽게 관리할 수 있도록 돕는 역할을 합니다.

🔍 쉬운설명

운영체제(OS)란 무엇일까요? 운영체제는 컴퓨터, 스마트폰, 태블릿 같은 모든 컴퓨팅 기계를 쉽게 사용할 수 있도록 도와주는 시스템 소프트웨어예요. 마치 우리 몸의 '뇌'나 회사의 '사장님'처럼, 하드웨어(물리적인 기기)와 소프트웨어(앱/프로그램) 사이에서 모든 것을 총괄하고 관리하는 역할을 합니다.

운영체제가 없다면? 만약 운영체제가 없다면, 우리는 컴퓨터를 켜고 끄는 것조차 너무 어려울 거예요. 메모리를 어떻게 사용할지, 하드디스크에 파일을 어떻게 저장할지, 키보드 입력은 어떻게 처리할지 등 모든 것을 우리가 직접 0과 1의 기계어로 명령해야 할 거예요!

다양한 운영체제:

- PC: 윈도우(Windows), 맥OS(macOS), 리눅스(Linux)
- 스마트폰: 안드로이드(Android), iOS(아이폰)

이처럼 우리 주변에는 다양한 운영체제가 존재하며, 각각의 운영체제는 기기를 다루는 방식이나 사용자 경험이 조금씩 달라요.

스마트폰 카메라 앱 예시: 당신이 스마트폰으로 사진을 찍는 앱을 만든다고 상상해 보세요.

- **카메라 접근 권한:** 앱이 카메라를 사용하려면 스마트폰 카메라에 접근할 수 있는 권한이 필요해요.
- **OS의 역할:** 운영체제(iOS 또는 Android)는 개발자가 카메라에 접근할 수 있도록 특별한 코드(API)를 제공해요. 하지만 **iOS가 제공하는 카메라 코드와 Android가 제공하는 카메라 코드는 서로 다르답니다!**
- **개발자의 역할:** 그래서 개발자는 아이폰용 카메라 앱과 안드로이드폰용 카메라 앱을 만들 때, 각 운영체제에 맞는 다른 코드를 사용해서 개발해야 해요. 마치 한국 사람과 미국 사람이 대화할 때, 각각 한국어와 영어를 써야 하는 것과 비슷하죠.
- **사용자 제어:** 또한, 운영체제는 "이 앱이 카메라를 사용해도 될까요?"와 같이 사용자에게 권한 허용 여부를 묻는 팝업을 띄워줘서, 사용자가 자신의 기기를 안전하게 제어할 수 있도록 도와줍니다.

결론적으로 운영체제는 우리가 컴퓨터나 스마트폰을 편리하고 안전하게 사용할 수 있도록 하드웨어와 소프트웨어 사이에서 모든 것을 중재하고 관리하는 **없어서는 안 될 핵심 소프트웨어**라고 할 수 있어요!

2. [수업자료] 앱/웹 서비스를 위한 모든 것

우리가 사용하는 웹/앱의 시작

우리가 일상적으로 사용하는 웹/앱 서비스는 명함 촬영, 사진 정보 전송, DB 입력과 같은 여러 단계를 거쳐 디지털 명함을 생성하는 과정을 통해 전반적인 서비스 구성의 기반을 다집니다.

쉬운설명

우리가 매일 사용하는 앱이나 웹 서비스는 아주 간단한 과정부터 시작해요. 예를 들어, 여러분이 명함을 스캔해서 디지털 명함으로 만드는 앱을 생각해 보세요.

1. **명함 촬영:** 앱을 켜서 종이 명함을 카메라로 찍습니다. (이때 앱의 '카메라' 기능이 사용되죠.)
2. **명함 사진 정보 전송:** 찍은 명함 사진 데이터를 앱이 알아볼 수 있는 형태로 서버로 보냅니다.
3. **DB 직접 입력:** 서버는 이 사진에서 글자를 인식해서, 명함 정보를 데이터베이스(DB)에 저장해요. 사람이 직접 입력하는 것과 같다고 볼 수 있습니다.
4. **디지털 명함:** 이 저장된 정보를 바탕으로 앱에서 언제든지 볼 수 있는 디지털 명함이 만들어집니다.

이런 일련의 과정들이 바로 우리가 웹/앱 서비스를 만들기 위해 알아야 할 기본적인 구성 요소들이에요. 이 과정들을 이해하면 복잡한 웹/앱 서비스도 쉽게 파악할 수 있답니다!

웹 주소의 시작, HTTP의 정체

통신(Communication)은 컴퓨터와 컴퓨터, 스마트폰과 서버 등 기기 간의 대화를 의미하며, **요청(Request)** 과 **응답(Response)** 으로 이루어진 **1요청 1응답의 원칙**을 가집니다.

HTTP (Hypertext Transfer Protocol) 는 이러한 통신 방식에 특정 규칙을 적용한 프로토콜로, 기기, 운영체제, 웹 브라우저에 상관없이 문자로 통신할 수 있는 방식을 의미합니다.

요소	설명
Hyper	기기(스마트폰)에 상관없이 문자로 통신할 수 있는 방식

요소	설명
Text	운영체제와 관계없이 문자로 통신할 수 있는 방식
Transfer	웹 브라우저에 상관없이 통신할 수 있는 방식
Protocol	통신 방식에 있어서 특정한 룰을 가지고 이대로 통신을 하자는 약속된 형태의 방식

Hypertext는 정보를 구조화하여 주고받는 형태를 의미하며, **JSON**과 **XML**이 대표적인 예시입니다.

파싱(Parsing): 문자를 특정한 형태로 잘라서 필요에 따라 사용하는 것을 의미합니다.

쉬운설명

우리가 인터넷을 사용할 때 컴퓨터나 스마트폰이 서로 정보를 주고받는 것을 "**통신**" 이라고 해요. 마치 두 사람이 대화하는 것과 같아요. 한 사람이 "**요청**" 하면 다른 한 사람은 그에 대한 "**응답**" 을 하죠. 예를 들어, 배달 앱에서 치킨을 주문하는 것이 '요청'이고, 치킨집에서 "주문 OK! 20분 기다려줘~"라고 답하는 것이 '응답'입니다.

모든 통신은 기본적으로 1요청, 1응답이라는 중요한 규칙을 가지고 있어요.

이러한 통신을 할 때, 컴퓨터들이 서로 알아들을 수 있도록 약속된 언어가 필요한데, 그것이 바로 **HTTP(Hypertext Transfer Protocol)** 입니다.

- **Hyper:** 어떤 기기(스마트폰, 컴퓨터 등)를 사용하든 상관없이 데이터를 주고받을 수 있다는 뜻이에요.
- **Text:** 글자 형태로 데이터를 주고받는다는 의미입니다. 그래서 어떤 운영체제(윈도우, iOS, 안드로이드)를 사용하든 문제가 없어요.
- **Transfer:** 데이터를 전송한다는 의미입니다. 웹 브라우저(크롬, 사파리 등)에 상관없이 데이터를 보낼 수 있어요.
- **Protocol:** 데이터를 주고받을 때 지켜야 할 "**약속**" 이나 "**규칙**" 을 의미해요.

결국 HTTP는 "**어떤 기기나 운영체제, 브라우저를 사용하든 상관없이 약속된 규칙에 따라 글자(텍스트) 형태로 정보를 주고받는 방식**" 이라고 할 수 있죠.

이때 주고받는 '글자 형태의 정보'를 Hypertext라고 부르는데, 주로 **JSON**이나 **XML** 같은 형식으로 되어 있어요. 텍스트를 특정 규칙에 따라 잘라서 필요한 정보만 뽑아내는 과정을 "**파싱(Parsing)**" 이라고 합니다. 예를 들어, JSON 형식의 디지털 명함에서 이름, 회사명, 전화번호를 파싱해서 쉽게 분류하고 사용하는 것이죠.

WAS(Web Application Server)의 구조

WAS(Web Application Server) 는 웹/앱 서비스의 핵심 구조로, 클라이언트(웹 브라우저, 스마트폰)의 **요청(Request)** 을 받아 웹 서버(HTTP 통신)를 통해 백엔드(동작 코드)로 전달하고, 데이터베이스(각종 정보 저장)에서 정보를 처리하여 **응답(Response)** 하는 역할을 수행합니다. 이 과정 또한 **1요청 1응답의 원칙**을 따릅니다.

Endpoint는 웹 서버의 주소를 가리키는 문자열로, 브라우저에서 입력하는 **www.google.com**과 같은 URL을 의미합니다.

쉬운설명

WAS(Web Application Server) 는 웹/앱 서비스의 '두뇌'이자 '심장' 같은 존재예요. 여러분이 웹사이트나 앱을 사용할 때 일어나는 모든 복잡한 요청과 응답을 처리하는 핵심 시스템이죠.

간단하게 설명하자면 이런 식이에요.

- **클라이언트 (Client):** 여러분이 사용하는 웹 브라우저(크롬, 엣지 등)나 스마트폰 앱이 클라이언트예요. 여기서 어떤 기능을 "**요청(Request)**" 합니다. 예를 들어, "이 사진을 저장해줘!"라고 요청하는 거죠.
- **Web Server (웹 서버):** 클라이언트의 요청을 받아주는 첫 번째 관문이에요. 여기서 HTTP 통신을 통해 요청을 받습니다. 웹 서버는 '응답(Response)'이라는 형태로 다시 클라이언트에게 결과를 돌려줍니다.
- **Back-end (백엔드):** 웹 서버가 받은 요청을 실제로 처리하는 '동작 코드'가 있는 부분입니다. 예를 들어, 사진을 어떻게 저장할지, 어떤 정보로 가공할지 등의 복잡한 로직이 여기서 실행돼요.
- **DataBase (데이터베이스):** 처리된 정보나 필요한 정보를 영구적으로 저장하는 곳이에요. 명함 정보, 회원 정보, 상품 정보 등이 이곳에 저장됩니다.

이 모든 과정은 **1요청 1응답의 원칙**을 철저히 지킵니다. 즉, 요청이 있으면 반드시 응답이 돌아와야 한다는 거죠.

Endpoint는 웹 서버의 '주소'를 의미해요. 우리가 www.naver.com이나 www.google.com처럼 웹 브라우저 주소창에 입력하는 것이 바로 Endpoint입니다. 여러분이 명함 앱에서 명함 정보를 저장하려 할 때, 앱은 특정 Endpoint로 "명함 정보를 저장해줘!"라고 요청하고, 서버는 그 요청을 받아 데이터베이스에 저장한 후 "저장 완료!"라고 응답하는 것이죠.

Front-end (클라이언트)

Front-end는 사용자가 서비스와 처음 만나는 화면이자 **사용자 경험(UI/UX)** 을 담당하는 부분입니다.

Front-end의 주요 특징:

- **UI/UX (화면과 사용자 경험 구성):** 사용자가 서비스를 편리하게 이용하도록 시각적인 디자인과 상호작용을 설계합니다.
 - **UI (User Interface):** 사용자가 처음 만나는 화면으로, 디자인적 요소를 담고 사용자 서비스 사용 편의성을 위합니다. **GUI (Graphics User Interface)** 의 줄임말이며, 그래픽을 활용한 사용자 환경을 의미합니다.
 - **UX (User Experience):** 서비스의 성격에 따라 사용자에게 편리한 경험을 제공하는 전반적인 과정을 의미합니다.
- **독립적인 기능들:** 서버와의 통신 없이도 카메라 촬영 등 단독으로 수행할 수 있는 기능들을 포함합니다.
- **서버에 요청하기 위한 통신 기능:** **GET, POST, PUT/PATCH, DELETE**와 같은 HTTP 통신 함수를 사용하여 서버에 정보를 요청하고 전달합니다.
 - **GET:** 기존에 저장된 정보나 페이지를 읽어옵니다.
 - **POST:** 새로운 정보를 저장하고 전달하기 위해 서버에 보냅니다.
 - **PUT, PATCH:** 기존에 저장된 정보의 내용을 수정합니다. PUT은 보안 이슈로 PATCH가 보완책으로 사용되기도 합니다.
 - **DELETE:** 저장된 정보를 삭제합니다.
- **사용자 패턴 분석을 위한 Analytics 구현:** 사용자 데이터를 수집하고 분석하여 서비스 개선에 활용합니다.

반응형 화면은 다양한 크기의 스크린(PC, 태블릿, 스마트폰)을 모두 커버하며 동일한 정보 제공 환경을 제공하는 것을 목표로 합니다.

Grid System은 반응형 디자인을 위해 일관된 규칙성을 적용하고, 화면비 계산과 작은 화면에서의 정보 제공 손실을 고려하는 시스템입니다.

쉬운설명

Front-end는 쉽게 말해, 여러분이 앱이나 웹사이트를 열었을 때 '**눈에 보이는 모든 것**' 을 의미해요. 마치 식당으로 치면 예쁘게 꾸며진 인테리어, 잘 정리된 메뉴판, 그리고 음식을 주문받는 종업원의 친절하고 태도와 같아요.

프론트엔드의 주요 특징:

UI/UX (사용자 화면과 경험):

- **UI(User Interface):** 여러분이 처음 앱을 켜고 보일 때 보이는 로그인 화면, 버튼, 이미지 등 시각적인 모든 요소를 말해요. 마치 식당의 예쁜 메뉴판이나 깔끔한 테이블 세팅과 같아요.
- **UX(User Experience):** 단순히 예쁜 것뿐만 아니라, 앱을 사용하는 전반적인 과정이 얼마나 편리하고 만족스러운지를 의미해요. 로그인 후 메인 화면에서 원하는 기능을 쉽게 찾고 실행할 수 있도록 설계하는 것이죠. 예를 들어, 식당에서 주문부터 식사, 결제까지 모든 과정이 불편함 없이 매끄럽게 진행되는 것과 같아요.

독립적인 기능: 웹사이트나 앱이 서버와 연결되지 않아도 자체적으로 할 수 있는 기능들이 있어요. 예를 들어, 명함 앱에서 명함을 '촬영'하는 기능은 인터넷이 없어도 카메라 앱처럼 작동하죠.

서버와의 통신: 프론트엔드는 서버에 필요한 정보를 '**요청**' 하고, 서버로부터 '**응답**' 을 받아요. 이때 사용하는 언어가 바로 HTTP 통신 함수들입니다.

- **GET:** "이 정보 좀 보여줘!" (예: 쇼핑몰에서 상품 상세 페이지 보기)
- **POST:** "새로운 정보 좀 저장해줘!" (예: 명함 정보를 새로 입력해서 저장)
- **PUT/PATCH:** "기존 정보 좀 수정해줘!" (예: 비밀번호 변경)
- **DELETE:** "이 정보 좀 삭제해줘!" (예: 계정 탈퇴)

사용자 패턴 분석 (Analytics): 사용자들이 앱을 어떻게 사용하는지 데이터를 모아서, 더 좋은 서비스를 만들기 위해 분석하는 기능이에요.

또한, 요즘에는 컴퓨터, 태블릿, 스마트폰 등 **어떤 기기에서 보더라도 화면이 알아서 조절되어 보기 좋게 나타나는 "반응형 화면"** 이 필수적이에요. 이를 위해 Grid System이라는 규칙을 사용해서 디자인하는데, 작은 화면에서도 정보가 잘 보이고 사용하기 편리하도록 만드는 것이 중요합니다.

동기와 비동기

동기(Synchronous) 방식은 **통신 결과와 함께 움직이는 방식**으로, 요청 후 응답이 올 때까지 다음 작업을 기다리는 직렬 처리 방식입니다.

예시: 강의 목록에서 상세 페이지로 이동, 저장된 명함 정보를 가져오는 페이지, 일반적인 웹 페이지의 이동, 정보와 함께 제공되어야 하는 화면.

비동기(Asynchronous) 방식은 **통신 결과와 상관없이 움직이는 방식**으로, 요청 후 응답을 기다리지 않고 다음 작업을 이어서 처리하는 병렬 처리 방식입니다.

예시: 페이지 이동 없이 댓글 달기, SNS 피드백, 디지털 명함이 없어도 영업 활동이 독립적으로 가능한 경우.

쉬운설명

동기와 비동기는 컴퓨터가 일을 처리하는 방식의 차이를 나타내는 중요한 개념이에요. 쉽게 말해, **일을 순서대로 처리할지 아니면 여러 일을 동시에 처리할지** 를 결정하는 방식입니다.

동기 (Synchronous)

"통신의 결과와 함께 움직임" 이라는 말이 핵심이에요. 하나의 작업이 완료될 때까지 다음 작업을 시작하지 않고 기다리는 방식입니다.

예시: 여러분이 식당에서 음식을 주문하고(요청), 그 음식이 나올 때까지 아무것도 하지 않고 기다리는 것과 같아요. 음식이 나와야 다음 행동(식사)을 할 수 있죠. 웹사이트에서 강의 목록을 클릭했을 때, 상세 페이지가 완전히 로드될 때까지 다른 페이지로 이동하거나 다른 작업을 할 수 없는 경우가 동기 방식의 예시입니다.

- **장점:** 작업의 순서가 명확하고 예측 가능해서 오류를 찾기 쉽습니다.
- **단점:** 한 작업이 오래 걸리면 전체 시스템이 멈춘 것처럼 느껴질 수 있어 비효율적일 수 있습니다.

비동기 (Asynchronous)

"통신의 결과와 상관없이 움직임"이라는 말이 핵심이에요. 요청을 보낸 후에 응답을 기다리지 않고 바로 다음 작업을 시작하는 방식입니다. 나중에 응답이 오면 그때 그 결과를 처리해요.

예시: 식당에서 음식을 주문하고(요청), 음식이 나올 때까지 기다리는 대신 다른 음료를 마시거나 다른 일(핸드폰 보기)을 하면서 기다리는 것과 같아요. 주문한 음식이 나오면 그때 받아서 먹는 거죠. 웹사이트에서 페이지 이동 없이 댓글을 다는 경우, 댓글을 작성하고 '등록' 버튼을 눌러도 페이지가 새로고침 되지 않고 바로 댓글이 추가되는 경우가 비동기 방식의 예시입니다.

- **장점:** 여러 작업을 동시에 처리할 수 있어 효율적이고, 사용자가 기다리는 시간을 줄여줍니다.
- **단점:** 작업 순서가 복잡해질 수 있고, 오류가 발생했을 때 추적하기 어려울 수 있습니다.

동기 방식은 '순차 처리', 비동기 방식은 '병렬 처리' 또는 '동시 처리'라고 생각하면 이해하기 쉽습니다. 웹/앱 서비스에서는 사용자의 편의를 위해 비동기 방식을 많이 사용하고 있습니다.

백엔드와 API

백엔드(Back-end)는 서버 측에서 동작하는 코드를 의미하며, **비즈니스 로직(Business logic)**을 처리하는 역할을 합니다.

비즈니스 로직: 컴퓨터 프로그램에서 데이터를 생성, 표시, 저장, 변경하는 모든 내용을 지칭합니다.

백엔드의 예시로는 명함 정보의 저장 및 변경, 쇼핑몰에서의 상품 구매 및 결제, SNS 회원 가입 등이 있습니다.

백엔드에서의 통신 역시 프론트엔드와 동일하게 HTTP 통신 함수를 사용합니다:

- **GET:** 기존의 저장된 정보, 페이지를 읽어온다.
- **POST:** 정보의 저장 및 전달을 위해 서버에 전달하는 역할.
- **PUT, PATCH:** 기존의 저장된 정보의 내용을 수정하는 역할.
- **DELETE:** 저장된 정보를 삭제.

프론트엔드와 백엔드는 동일한 HTTP 메서드로 통신해야 합니다. 즉, 프론트엔드가 GET으로 요청하면 백엔드도 GET으로 응답해야 합니다.

REST API는 약속된 주소(URI)를 가지고 있고, 각 주소에 기능(함수)을 가지고 있는 형태로 프론트엔드와 백엔드가 정보를 주고받는 표준화된 방식입니다.

- **URI (Uniform Resource Identifier):** 서버에서의 특정 리소스(자원) 식별자를 지칭합니다.
- **URL (Uniform Resource Locator):** 우리가 흔히 사용하는 웹 사이트의 주소를 뜻하며, URI의 한 종류입니다.

쉬운설명

백엔드(Back-end)는 웹/앱 서비스에서 '눈에 보이지 않는 곳'에서 일어나는 모든 중요한 작업들을 처리하는 부분이에요. 마치 식당의 주방과 같아요. 손님들에게는 보이지 않지만, 주문받은 음식을 요리하고, 식재료를 관리하고, 재고를 파악하는 등 핵심적인 일들이 모두 주방에서 이루어지죠.

백엔드의 핵심 역할은 **비즈니스 로직(Business Logic)**을 처리하는 것입니다.

비즈니스 로직이란 데이터를 만들고(생성), 보여주고(표시), 저장하고(저장), 바꾸는(변경) 등 서비스의 운영에 필요한 모든 규칙과 과정을 의미해요. 예시: 여러분이 명함 앱에서 새 명함을 등록하면, 백엔드는 그 명함 정보를 받아서 데이터베이스에 저장하고, 필요한 경우 가공하는 로직을 수행합니다. 쇼핑몰에서 상품을 구매하고 결제하는 과정, SNS에서 회원가입을 하는 것 등 복잡한 데이터 처리와 규칙 적용은 모두 백엔드가 담당해요.

프론트엔드와 마찬가지로 백엔드도 클라이언트와 정보를 주고받을 때 HTTP 통신 함수를 사용해요.

- **GET**: 정보를 '가져오는' 요청
- **POST**: 새로운 정보를 '만들어 저장하는' 요청
- **PUT/PATCH**: 기존 정보를 '수정하는' 요청
- **DELETE**: 정보를 '삭제하는' 요청

여기서 **중요한 점은 프론트엔드가 어떤 HTTP 메서드로 요청했으면, 백엔드도 그 메서드에 맞춰 응답해야 한다는 약속입니다.** 예를 들어, 프론트엔드가 '명함 정보를 가져와!'라고 **GET** 요청을 보냈으면, 백엔드도 **GET** 요청에 맞는 방식으로 정보를 찾아 응답해야 하는 거죠.

이러한 약속된 통신 방식을 **REST API**라고 부릅니다.

- **API (Application Programming Interface)** 는 프로그램들이 서로 소통할 수 있도록 만들어진 '접점' 또는 '규칙'이에요.
- **REST (Representational State Transfer)** 는 API를 만드는 한 가지 '스타일' 또는 '원칙'입니다.

REST API는 마치 표준화된 통신 수단과 같아요. <https://school.elice.io/courses/26028/info>와 같은 **약속된 주소(URI)** 를 통해, 각 주소가 특정 기능(함수)을 수행하도록 설계되어 있습니다.

URI는 특정 '자원'을 식별하는 고유한 이름이고, URL은 그 자원의 '위치'를 나타내는 주소입니다. **URL은 URI의 한 종류** 라고 생각하시면 됩니다.

백엔드와 API는 웹/앱 서비스가 원활하게 작동하도록 하는 숨겨진 영웅들이라고 할 수 있어요!

3. [수업자료] 데이터베이스와 REST API

데이터베이스란?

데이터베이스는 **기록하고 조합하는 모든 데이터들의 총합**을 의미합니다. 일상생활에서 은행 거래 내역 조회, 도서관 도서 검색 등 다양한 곳에서 활용됩니다. 은행 결제 내역을 조회하는 과정은 앱 실행 → 서버 접속 → 접속 정보 확인 → 데이터베이스 검색 → 결과 확인의 4단계로 이루어집니다. 도서관 관리 프로그램은 책의 보유 여부, 정보(제목, 작가, 출판사), 대여 현황 및 대여자, 반납 예정일 등의 정보를 필요로 합니다.

쉬운설명

데이터베이스(DB) 는 무엇일까요? 간단히 말해, 데이터베이스는 우리가 필요로 하는 모든 정보를 **체계적으로 모아두고, 관리하며, 쉽게 찾아 쓸 수 있도록 정리해 놓은 거대한 '정보 창고'** 예요.

일상생활 속 DB 예시:

은행 앱으로 결제 내역 확인:

1. 우리가 은행 앱을 열면
2. 앱은 은행 서버에 "홍길동 계좌 정보를 보여줘!" 하고 요청해요.

3. 은행 서버는 우리가 진짜 홍길동인지 확인하고, 홍길동의 계좌 정보를 DB에서 찾아와요.
4. 마지막으로 우리가 앱에서 원하는 결제 내역을 볼 수 있게 되는 거죠!

여기서 중요한 건, 우리의 **모든 거래 내역이 은행의 데이터베이스에 차곡차곡 쌓여 있다**는 거예요.

도서관에서 책 찾기:

도서관에는 수많은 책이 있죠? 어떤 책이 있는지, 누가 빌려갔는지, 언제 반납되는지 등 모든 정보가 도서관 데이터베이스에 저장되어 있어요. 우리가 원하는 책을 검색하면, 도서관 시스템이 DB에서 해당 책 정보를 빠르게 찾아 보여주는 거예요.

이렇게 데이터베이스는 우리 주변의 수많은 서비스가 원활하게 작동하도록 돕는 아주 중요한 역할을 합니다!

데이터베이스 기능

데이터베이스의 기본 기능은 **저장(Insert)**, **검색(Select/Find)**, **수정(Update)**, **삭제(Delete)** 네 가지입니다.

저장 (Insert)

데이터를 기록하는 기능입니다.

- 예: 상품의 사진, 가격, 상세설명 등 판매에 필요한 정보를 테이블 형태로 저장합니다.
- 회원 정보도 상품번호, 이름, 아이디, 주소 등으로 분류하여 저장할 수 있습니다.
- **데이터를 분류하기 좋게 저장하는 것이 중요하며, 이는 서비스를 운영하는 핵심 요소입니다.**
- 문제 상황 예시: 신발이 매진되었을 때, 단순히 상품 조회를 하면 결제 과정에서 판매 수량이 없어 환불 로직을 추가해야 하는 문제가 발생할 수 있습니다.

검색 (Select, Find)

저장된 데이터 중에서 필요한 정보를 찾아내는 기능입니다.

- 특정 조건을 활용해 저장된 데이터들을 분류하여 보여줄 수 있습니다.
- 예: 신상품 목록 조회 시, 신상품 테이블에서 신상품만 저장하고 전체 목록을 조회하여 결과를 반환합니다.
- **Query**는 데이터베이스에 명령을 내리는 언어입니다.
- 데이터는 웹 브라우저/스마트폰 → 서버 조회 요청 → DB 쿼리 조회 → 조회 결과 전달의 과정을 거쳐 우리에게 옵니다.

수정 (Update)

저장된 데이터의 내용을 변경하는 기능입니다.

- 데이터 타입에는 숫자(**Int**, **float**, **double**), 문자(**String**), 참/거짓(**Boolean**), 시간(**Time**, **Datetime**) 등이 있습니다.
- 문제 상황 해결 예시: 신발이 매진되었을 때, 매진이라는 새로운 속성(**Boolean** 타입)을 추가하여 **True**로 변경하면, 상품 조회 단계에서 미리 매진 여부를 확인하여 불필요한 결제 과정을 막을 수 있습니다.

삭제 (Delete)

데이터베이스에서 특정 데이터를 제거하는 기능입니다.

- 데이터를 완전히 제거하는 **물리 삭제**와 데이터는 남겨두지만 특정 플래그를 통해 '삭제됨'으로 표시하는 **논리 삭제**가 있습니다.

- 논리 삭제 예시: 신발이 매진되었을 때, 매진 필드를 **True**로 설정하여 사용자에게는 보이지 않지만 실제 데이터는 남아있게 합니다.

데이터베이스는 원하는 목적에 맞게 초기에 구성을 잘 하는 것이 중요합니다.

🔍 쉬운설명

데이터베이스의 4가지 기본 기능 (CRUD) 데이터베이스는 마치 도서관과 같다고 생각할 수 있어요. 책(데이터)을 넣고 (저장), 찾고(검색), 내용을 고치고(수정), 버리는(삭제) 기본적인 작업들이 있죠. 컴퓨터에서는 이것 **CRUD**라고 부르는데, **Create**(생성), **Read**(읽기), **Update**(수정), **Delete**(삭제)의 약자예요.

저장 (Insert) - 책꽂이에 새 책 꽂기 📖

새로운 데이터를 데이터베이스에 추가하는 기능이에요. 예시: 온라인 쇼핑몰에 새 상품 '예쁜 티셔츠'가 들어오면, 이 티셔츠의 사진, 가격, 이름, 상세 설명 같은 정보를 데이터베이스에 입력해서 저장하는 거죠. **Tip:** 데이터를 저장할 때는 나중에 찾기 쉽게 **정리된 형식(테이블)** 으로 저장하는 게 아주 중요해요. 마치 도서관에서 책을 주제별, 작가별로 분류해서 꽂는 것처럼요!

검색 (Select, Find) - 원하는 책 찾아보기 🔍

데이터베이스에 저장된 수많은 정보 중에서 우리가 필요한 정보를 찾아내는 기능이에요. 예시: 쇼핑몰에서 '신상품'을 보고 싶으면, 데이터베이스에 "최근에 들어온 상품들만 보여줘!" 하고 요청하는 거죠. 이때 사용하는 명령어를 **쿼리 (Query)** 라고 해요. 마치 도서관 사서에게 "SF 소설 중에 가장 인기 있는 책 찾아주세요!" 하고 말하는 것과 같아요.

데이터가 우리에게 오려면: 나(앱/웹) → 서버 → DB에 쿼리 요청 → DB에서 결과 전달 → 서버 → 나(앱/웹) 이런 복잡한 과정을 거쳐요!

수정 (Update) - 책 내용 고치기 ✏️

이미 저장된 데이터의 내용을 바꾸는 기능이에요. 예시: '예쁜 티셔츠'의 가격이 5만 원에서 4만 원으로 할인되면, 데이터베이스에 있는 티셔츠 가격 정보를 4만 원으로 업데이트하는 거죠.

매진된 상품 처리: 이 기능은 특히 중요해요. 예전에 신발이 매진되었을 때, 단순히 품절이라고만 해두면 고객이 구매 버튼을 누르고 나서야 "품절입니다"라고 뜨는 불편함이 있었죠. 이제는 매진이라는 새로운 정보(데이터 타입: **Boolean**, 참/거짓)를 추가해서, 신발이 매진되면 이 정보를 **True**로 바꿔줘요. 그러면 고객이 상품을 조회할 때부터 "매진입니다"라고 미리 알려줄 수 있어서 훨씬 편리해져요!

삭제 (Delete) - 낡은 책 버리기 🗑️

더 이상 필요 없는 데이터를 데이터베이스에서 제거하는 기능이에요. 예시: 더 이상 생산되지 않는 '오래된 청바지' 상품 정보를 삭제할 수 있어요.

- **물리 삭제:** 완전히 없애버리는 거예요. (진짜 책을 폐기하는 것)
- **논리 삭제:** 데이터는 그대로 두되, '이 데이터는 삭제된 것이나 다름없다'고 표시만 해두는 거예요. (책은 창고에 두지만, 검색 시스템에서는 안 보이게 하는 것) 논리 삭제는 나중에 데이터 복구가 필요할 때 유용해요.

이런 기본 기능들을 잘 활용하고, **데이터베이스를 목적에 맞게 처음부터 잘 설계하는 것이 효율적인 서비스 운영의 핵심이에요!**

데이터베이스와 서버

서버 오류는 프론트엔드(사용자 화면)와 백엔드(서버) 간의 통신이 불가능해진 상황을 의미합니다.

서버 오류의 다양한 원인:

- **트래픽 과부하:** 서버에 접속하는 통신 양(트래픽)이 너무 많아 서버가 요청을 처리하지 못하는 경우입니다.
- **백엔드 코드의 버그:** 서버를 구성하는 코드에 오류가 있는 경우입니다.
- **데이터베이스 접속 오류:** 서버에 접속은 가능하지만, 서버가 데이터베이스와 연결되지 못하는 경우입니다.

쉬운설명

서버가 터졌어요! (데이터베이스와 서버 오류) 온라인 서비스를 이용하다가 갑자기 화면이 멈추거나 '페이지를 찾을 수 없습니다' 같은 메시지를 본 적 있나요? 이걸 보통 "서버가 터졌다"고 하죠.

서버 오류란 프론트엔드와 백엔드 사이의 '대화'가 끊어지거나 불가능해진 상황을 의미해요. 마치 여러분이 친구에게 메시지를 보냈는데, 친구 폰이 꺼져서 답장을 받을 수 없는 것과 같아요.

서버 오류가 발생하는 다양한 이유:

트래픽 과부하 (Traffic Overload):

트래픽은 서버에 접속하려는 사람들의 통신량을 의미해요. 예시: 인기 아이돌 콘서트 티켓 예매가 시작되는 순간, 수십만 명이 동시에 웹사이트에 몰리면 서버는 이 모든 요청을 한 번에 처리하기 어려워져요. 마치 좁은 문으로 너무 많은 사람이 한꺼번에 들어가려고 하면 문이 막히는 것과 같죠. **서버가 감당할 수 있는 양을 초과하면, 결국 시스템이 다운되거나 속도가 극도로 느려지는 현상이 발생해요.**

백엔드 코드의 버그 (Bug in Backend Code):

백엔드 서버를 작동시키는 프로그램 코드에 오류(버그)가 있을 수 있어요. 예시: 개발자가 특정 기능을 만들었는데, 그 코드 안에 계산 실수가 있거나 예상치 못한 상황에 대한 처리가 빠져있을 경우, 그 기능이 실행될 때마다 서버가 멈출 수 있어요. 마치 로봇이 특정 명령을 수행하다가 이해하지 못해서 멈춰버리는 것과 같아요.

데이터베이스 접속 오류 (Database Connection Error):

서버 자체는 잘 작동하고 있지만, 서버가 데이터를 가져와야 할 데이터베이스(정보 창고)와 연결되지 못하는 상황이에요. 예시: 쇼핑몰 서버는 정상적으로 돌아가고 있는데, 서버가 '상품 재고' 정보를 얻으려고 데이터베이스에 접근하려 했으나, 데이터베이스 서버가 꺼져 있거나 연결 통로에 문제가 생겨서 정보를 가져올 수 없는 거죠. 결국 서버는 고객에게 아무런 정보도 보여줄 수 없게 돼요.

이러한 오류들을 줄이고 안정적인 서비스를 제공하기 위해 개발자들은 서버를 튼튼하게 만들고, 문제가 생겼을 때 빠르게 해결할 수 있는 시스템을 구축하는 데 많은 노력을 기울입니다!

데이터베이스와 웹 서버의 연결

클라이언트(웹 브라우저/앱)는 웹 서버에 요청을 보냅니다. 웹 서버는 백엔드 코드를 통해 데이터베이스에 필요한 데이터를 요청하고, 데이터베이스는 응답하여 웹 서버로 데이터를 보냅니다. 웹 서버는 이 데이터를 다시 클라이언트로 전달하여 화면에 표시하게 됩니다. 이 과정에서 **웹 서버와 데이터베이스는 서로 긴밀하게 연결되어 정보를 주고받습니다.**

쉬운설명

데이터베이스와 웹 서버는 어떻게 연결되어 있을까요? 우리가 웹사이트나 앱을 사용할 때, 눈에 보이지 않는 곳에서 웹 서버와 데이터베이스가 끊임없이 대화하며 정보를 주고받고 있어요. 이 과정을 간단히 설명해 드릴게요.

클라이언트 (나) → 웹 서버 (식당 종업원):

여러분은 스마트폰이나 컴퓨터(클라이언트)에서 웹사이트에 접속해요. (예: "저녁 메뉴판 보여주세요!") 여러분의 요청은 인터넷을 통해 웹 서버로 전달돼요. 웹 서버는 마치 식당에서 주문을 받는 종업원과 같아요.

웹 서버 → 백엔드 코드 (주문표 작성) → 데이터베이스 (주방 재료 참고):

웹 서버는 여러분의 요청을 받으면, 미리 작성된 백엔드 코드를 통해 어떤 정보를 가져와야 할지 파악해요. 그다음, 백엔드 코드는 데이터베이스(DB)에게 "메뉴판에 필요한 음식 정보 좀 주세요!"라고 요청해요. 데이터베이스는 식당의 모든 재료와 요리법이 담긴 주방 재료 참고 같은 곳이지요.

데이터베이스 → 웹 서버 (참고에서 재료 꺼내 종업원에게 전달):

데이터베이스는 백엔드 코드의 요청에 따라 필요한 데이터(메뉴 정보)를 찾아 웹 서버로 보내줘요.

웹 서버 → 클라이언트 (종업원이 나에게 메뉴판 전달):

웹 서버는 데이터베이스로부터 받은 정보를 가공하여 여러분의 웹 브라우저나 앱 화면에 표시할 수 있는 형태로 만드어요. 그리고 이 정보를 다시 여러분의 클라이언트(스마트폰/컴퓨터)로 보내주죠. 그러면 여러분은 화면에서 맛있는 음식 메뉴를 볼 수 있게 되는 거예요!

이처럼 웹 서버는 클라이언트의 요청을 받아 데이터베이스와 소통하고, 그 결과를 다시 클라이언트에게 전달하는 **중간 다리 역할**을 합니다. 이 연결이 원활해야 서비스가 문제없이 작동할 수 있어요.

API와 데이터베이스

API (Application Programming Interface) 는 프로그램끼리(Code끼리) 대화를 하도록 도와주는 일을 말합니다. Application, Programing, Interface의 약자입니다.

- **API 요청:** 예를 들어, 로그인 시 ID/PW를 입력하면 로그인 API가 호출되고, 이 API는 로그인 정보를 데이터베이스에서 조회하여 성공 여부를 반환합니다.
- **로그인 API:** [Shoppingmall.com/login](https://www.shoppingmall.com/login)과 같은 endpoint로 아이디/비밀번호를 전송(**POST** 방식)하여 로그인 요청을 처리합니다.
- **GET API:** 웹사이트의 특정 정보를 가져올 때 사용됩니다.
 - <https://www.apple.com/kr/>는 한국어 사이트 정보를
 - <https://www.apple.com/jp/>는 일본어 사이트 정보를 가져오는 예시입니다.
 - 이처럼 <https://www.apple.com/국가코드/>와 같이 국가코드를 통해 다른 언어의 정보를 가져올 수 있습니다.
- API는 **네트워크를 통한 기능 호출**을 가능하게 합니다.
- **Open API:** 행정안전부에서 제공하는 실시간 주소정보 조회(검색API)와 같이 누구나 사용할 수 있도록 공개된 API를 통해 다양한 서비스를 만들 수 있습니다.

쉬운설명

API는 프로그램들의 통역사! API는 "Application Programming Interface"의 줄임말이에요. 쉽게 말해, 서로 다른 두 프로그램이 대화할 수 있도록 연결해 주는 '다리' 이자 '통역사' 역할을 해요.

API는 왜 필요할까요?

예시: 우리가 식당에 가서 "돈가스 주세요"라고 주문하면, 종업원이 주방(백엔드)에 가서 주방장(데이터베이스)에게 "돈가스 요청이요!"라고 전달하죠? 그리고 주방장이 돈가스를 만들면 종업원이 다시 우리에게 가져다주고요. **여기서 종업원이 바로 API와 비슷하다고 생각하면 돼요.**

API의 작동 방식:

로그인 API:

우리가 쇼핑몰에서 ID와 비밀번호를 입력하고 '로그인' 버튼을 누르면, 우리의 앱/웹(클라이언트)은 로그인 API를 호출해요. 이 API는 우리의 ID/PW 정보를 서버로 안전하게 전달하고, 서버는 이 정보를 데이터베이스에 저장된 회원 정보와 비교해서 "이 사용자가 맞는지" 확인해요. 확인이 되면, API는 우리에게 "로그인 성공!"이라는 응답을 보내주는 거죠. 이때, [Shoppingmall.com/login](https://www.shoppingmall.com/login)과 같은 주소(이를 **엔드포인트(Endpoint)** 라고 해요)로 ID/PW를 보내는데, 이 방식이 바로 **POST** 방식이에요.

GET API (정보 가져오기):

웹사이트에서 단순히 정보를 '가져올' 때 많이 사용돼요. 예시: 애플 웹사이트에서 <https://www.apple.com/kr/>에 접속하면 한국 애플 웹사이트 정보를 가져오고, <https://www.apple.com/jp/>에 접속하면 일본 애플 웹사이트 정보를 가져오죠. 이처럼 <https://www.apple.com/국가코드/>와 같이 특정 규칙(국가코드)을 사용해서 원하는 정보를 쉽게 가져올 수 있게 해줘요.

Open API는 무엇인가요?

오픈 API는 말 그대로 누구나 자유롭게 사용할 수 있도록 공개된 API를 말해요. 예시: 행정안전부에서 '실시간 주소 정보'를 조회할 수 있는 오픈 API를 제공하는데, 개발자들은 이 API를 활용해서 내비게이션 앱이나 쇼핑몰 주소 입력 시스템 같은 서비스를 만들 수 있어요. 우리는 직접 전국 주소 데이터를 만들 필요 없이, 정부가 제공하는 API를 가져다 쓰는 거죠.

결론적으로 **API**는 프로그램 간의 효율적인 소통을 가능하게 해서, 우리가 사용하는 수많은 서비스가 원활하게 돌아가도록 돕는 핵심 기술입니다.

데이터베이스의 종류와 선택

DBMS (Database Management System) 는 데이터베이스 운영관리를 위한 소프트웨어입니다.

DBMS의 종류:

- **MySQL:** Oracle에서 제작 (가장 널리 사용)
- **MariaDB:** MariaDB 재단에서 제작 (MySQL에서 파생)
- **PostgreSQL:** PostgreSQL Global Development Group에서 제작
- **Oracle:** Oracle에서 제작 (기업 환경에서 많이 사용)

데이터베이스 종류 - 관계형 vs NoSQL

관계형 데이터베이스 (RDBMS):

데이터를 테이블 형태로 저장하고, 이 테이블 간의 관계를 **Primary Key(기본 키)** 와 같은 고유한 값을 활용하여 연결하는 데 유리합니다. 예: "B신발을 구매한 A고객"의 정보를 저장할 때, 'B신발 데이터'와 'A고객 데이터'를 각각의 테이블에 저장하고, 주문 정보 테이블에서 이 둘을 연결하여 관리합니다. **데이터의 일관성과 정합성이 중요한 금융, 재고 관리 등에 적합합니다.**

NoSQL 데이터베이스:

관계형 데이터베이스와 달리 테이블 스키마가 정해져 있지 않아 유연하게 데이터를 저장할 수 있습니다. 기록이 단순하고 대용량 처리에 유리하며, 데이터 모델이 다양합니다 (문서, 키-값, 컬럼, 그래프 등). 예: 소셜 미디어의 게시물, 로그 데이터, 실시간 웹 분석 데이터와 같이 끊임없이 생성되고 구조가 유동적인 데이터를 처리하는 데 적합합니다.

쉬운설명

데이터베이스의 종류: DBMS와 관계형 vs NoSQL

데이터베이스를 효율적으로 관리하려면 '데이터베이스 관리 시스템'이라는 특별한 소프트웨어가 필요해요.

DBMS (Database Management System) - 데이터베이스를 관리하는 소프트웨어

DBMS는 여러분이 데이터베이스에 데이터를 저장하고, 찾고, 수정하고, 삭제하는 모든 작업을 도와주는 소프트웨어예요. 마치 컴퓨터의 운영체제(OS)가 컴퓨터 하드웨어를 관리해 주는 것처럼요.

대표적인 DBMS 종류:

- **MySQL:** 가장 널리 사용되는 DBMS 중 하나예요. 웹 서비스나 앱 개발에 많이 쓰여요. (오라클 제작)
- **MariaDB:** MySQL에서 파생된 것으로, MySQL과 비슷하게 작동하지만 오픈소스 커뮤니티에서 활발하게 개발되고 있어요.
- **PostgreSQL:** 기능이 강력하고 안정성이 높아 복잡한 데이터 처리에 강점을 보여요.
- **Oracle:** 대규모 기업 시스템에서 많이 사용하는 매우 강력하고 유료인 DBMS예요.

데이터베이스의 두 가지 큰 종류: 관계형 vs NoSQL

관계형 데이터베이스 (Relational Database)

특징: 데이터를 테이블 형태로 저장해요. 마치 엑셀 스프레드시트처럼 행과 열로 구성되어 있죠. 그리고 이 테이블들이 서로 '관계'를 맺고 있어요. '관계'의 의미: 예를 들어, '고객' 테이블과 '주문' 테이블이 있다면, '주문' 테이블에는 어떤 고객이 주문했는지 알 수 있도록 '고객 ID'가 기록돼요. 이 '고객 ID'가 바로 테이블들을 연결해 주는 **기본 키(Primary Key)** 역할을 해요.

- **장점:** 데이터의 정확성(일관성)과 무결성이 매우 중요할 때 좋아요. 은행의 계좌 관리나 쇼핑몰의 재고 관리처럼 꼼꼼하고 정확해야 하는 곳에 주로 사용돼요.
- **단점:** 테이블 구조를 미리 정해놔야 해서, 데이터 구조가 자주 바뀌는 경우 다루기 어려울 수 있어요.

NoSQL 데이터베이스 (Not only SQL Database)

특징: 관계형처럼 엄격한 테이블 구조를 따르지 않고, 데이터를 더 자유로운 형태로 저장해요. (예: 문서, 키-값 쌍, 그래프 등)

- **장점:** 데이터 구조가 유연해서 빠르게 변화하는 서비스에 적합해요. 대량의 데이터를 빠르게 처리하고 분산 처리하기 좋아요. (수십억 명의 사용자가 동시에 접속하는 SNS 같은 서비스에 유리)
- **단점:** 데이터의 일관성이나 관계를 관리하는 것이 관계형 DB보다 복잡할 수 있어요.
- **사용 예:** 페이스북의 게시물, 인스타그램의 사진, 실시간 게임 데이터, 로그 기록 등 형태가 다양하고 끊임없이 생성되는 데이터를 저장하고 분석하는 데 많이 쓰여요.

어떤 데이터베이스를 선택할까요? 이 두 가지 방식 중 어떤 것을 선택할지는 서비스의 목적, 처리해야 할 데이터의 종류와 양, 그리고 데이터의 중요성에 따라 달라져요. **정확성이 최우선이라면 관계형, 유연성과 대용량 처리가 중요하다면 NoSQL을 고려할 수 있어요.**

4. [수업자료] 라이브러리와 프레임워크

웹의 역사

웹의 역사는 프로그래밍 기술의 발전을 이해하는 데 중요한 배경이 됩니다. 초기 웹은 HTML을 통해 정보를 주고받는 형태로 시작되었으며, info.cern.ch는 세계 최초의 웹사이트로 기록되어 있습니다. **팀 버너스리** 경은 WWW(월드 와이드 웹)의 창시자로, HTML, CSS, HTTP 통신, URL 등 현대 웹의 초석이 되는 기술들을 개발하고 연구자 간 자료 공유를 위해 만들어진 웹을 대중에게 무료로 공개하여 인류 공헌을 인정받았습니다. 시간이 지나면서 웹 프로그래밍은 점차 커지고 복잡해졌고, 반복적인 코드 사용, 협업 프로젝트 증가 등으로 인해 구조화된 체계의 필요성이 대두되었습니다.

🔍 쉬운설명

웹의 역사는 우리가 지금 보고 있는 인터넷이 어떻게 발전해 왔는지를 보여주는 흥미로운 이야기예요.

초기 웹의 시작 (1990년대 초): 팀 버너스리 경이라는 한 과학자가 연구자들이 정보를 쉽게 공유할 수 있도록 '월드 와이드 웹(WWW)'을 만들었어요. 이때는 HTML이라는 언어를 써서 글과 그림을 보여주고, HTTP라는 규칙으로 정보를 주고받았죠. info.cern.ch는 세상에 처음 등장한 웹사이트랍니다. 초기 웹은 정말 단순했어요. 마치 옛날 흑백 TV처럼요!

웹의 성장과 복잡성 증가: 시간이 흐르면서 웹은 점점 더 많은 정보를 담고, 더 많은 기능들을 수행하게 되었어요. 단순한 정보 공유를 넘어 온라인 쇼핑, 소셜 미디어 등 다양한 서비스가 등장했죠. 네이버 같은 포털 사이트의 변화를 보면 웹이 얼마나 발전했는지 알 수 있어요.

새로운 필요성의 대두: 웹이 커지고 복잡해지면서 개발자들은 같은 코드를 반복해서 쓰거나, 여러 명이 함께 작업할 때 어려움을 겪기 시작했어요. "더 효율적이고, 체계적으로 코드를 만들 방법은 없을까?" 하는 고민이 생겨났죠. 바로 이 때문에 나중에 배우게 될 라이브러리와 프레임워크 같은 도구들이 중요해진 배경이 됩니다. **웹의 복잡성이 증가하면서 개발자들이 느끼는 어려움을 해결하기 위해 더 나은 개발 방법론과 도구들이 필요하게 된 것이죠.**

라이브러리

라이브러리는 **반복해서 자주 사용하는 코드들을 모아둔 일종의 도구 모음**입니다. 코드의 역사가 쌓이면서 자주 사용되는 기능들이 발생했고, 이를 함수로 묶어 선언하여 사용하는 형태로 발전했습니다. pub.dev와 같은 플랫폼에서 다양한 라이브러리(package라고도 불림)를 찾아볼 수 있으며, 오픈소스 형태로 활발하게 생산 및 공유됩니다.

- **예시:** [Chart.js](https://www.chartjs.org/)는 간단한 데이터 입력만으로 복잡한 차트를 쉽게 그려낼 수 있게 해주는 라이브러리입니다. [Passport.js](https://passportjs.org/)는 로그인 및 회원가입 관리, 소셜 로그인 구현 등을 쉽게 도와주는 라이브러리로, Kakao나 Naver용 passport도 존재합니다.
- **사용 이유:** 검증된 라이브러리를 잘 활용하는 것은 개발자의 능력이 됩니다. 데이터 처리와 같이 학습이 필요한 큰 라이브러리도 있으며, 부적절하게 사용하면 성능 저하를 일으킬 수도 있습니다. 궁극적으로 라이브러리는 **프로젝트의 생산성 향상과 효율성 증진**을 위해 활용됩니다.

라이브러리 vs API:

- **공통점:** 재사용 가능한 코드의 반복적 활용.
- **차이점:** API는 네트워크 통신을 통해 기능을 제공하는 반면, 라이브러리는 직접 코드에 포함되어 사용됩니다.

🔍 쉬운설명

라이브러리는 **"자주 쓰는 기능들을 모아놓은 상자"** 라고 생각하면 이해하기 쉬워요!

왜 필요할까요? 개발을 하다 보면 "로그인 기능", "데이터 그래프 그리기"처럼 비슷한 기능을 여러 프로젝트에서 계속 만들게 돼요. 매번 처음부터 코드를 짜는 건 너무 번거롭고 시간 낭비겠죠?

라이브러리의 역할: 이런 반복적인 작업들을 미리 잘 만들어진 코드 덩어리(함수)로 묶어서 제공하는 것이 라이브러리에요. 개발자는 필요할 때 이 상자에서 원하는 도구(기능)를 꺼내 쓰기만 하면 돼요. 예를 들어, [Chart.js](https://www.chartjs.org/)라는 라이브러리

는 복잡한 그래프를 단 몇 줄의 코드로 만들 수 있게 해줘요. **Passport.js**는 카카오톡이나 네이버 로그인처럼 복잡한 인증 과정을 쉽게 구현하게 해주고요!

어디서 찾을 수 있나요? **pub.dev** 같은 웹사이트에 가면 수많은 라이브러리들이 공유되어 있어요. 전 세계 개발자들이 만든 유용한 도구들이 모여 있는 곳이죠.

개발자의 능력 향상: 좋은 라이브러리를 찾아서 프로젝트에 잘 적용하는 것도 개발자의 중요한 능력 중 하나예요. 하지만 무턱대고 쓰기만 하면 오히려 프로그램이 느려질 수도 있으니, 적절하게 사용하는 지혜가 필요합니다!

API와의 차이점: 라이브러리는 여러분이 직접 코드를 가져와서 내 프로그램 안에 넣고 사용하는 반면, API는 다른 컴퓨터(서버)에 있는 기능을 "요청"해서 받아 쓰는 방식이에요. 예를 들어, 네이버 날씨 API는 네이버 서버에 "오늘 날씨 알려줘"라고 요청하면 그 정보를 받아오는 식이죠. **둘 다 재사용 가능한 기능을 제공하지만, 작동 방식에 차이가 있습니다.**

프레임워크

프레임워크는 **프로그램을 효율적으로 짜기 위한 '틀(Frame)'**을 제공합니다. 빵 모양을 만들기 위한 빵틀처럼 코드를 작성하기 위한 구조를 제시하여, 개발자가 코드 로직 자체에 집중할 수 있게 도와줍니다.

특징:

- 공통된 규칙을 활용해야 하므로 **협업에 매우 유리합니다.**
- 언어별로 다양한 프레임워크가 존재하며, 각각 제공하는 도구나 틀이 다릅니다.
- **IoC (Inversion of Control)** 개념을 포함하여, 개발자가 아닌 프레임워크가 전체적인 흐름을 제어합니다.

예시: **Django**는 파이썬 기반의 웹 페이지를 작성할 때 사용되는 대표적인 프레임워크입니다. 일반적으로 관리 페이지를 따로 작성해야 하지만, Django는 상당한 수준의 관리자 페이지를 기본적으로 제공하여 개발자의 제작 부담을 덜어줍니다.

쉬운설명

프레임워크는 "건물을 지을 때 필요한 뼈대와 설계도" 같아요!

왜 필요할까요? 라이브러리가 특정 기능(문 만들기, 창문 달기)을 대신 해주는 도구라면, 프레임워크는 건물의 전체적인 구조(방은 어디에, 화장실은 어디에)를 미리 정해놓은 큰 틀이에요. 이 틀 안에서 개발자들은 정해진 규칙에 따라 코드를 작성하면 돼요.

프레임워크의 장점:

- **협업 효율성:** 여러 명이 함께 개발할 때, 모두가 같은 틀과 규칙을 따르기 때문에 코드가 일관되고 이해하기 쉬워요. 마치 모든 건축가가 같은 설계도를 보고 작업하는 것과 같죠.
- **생산성 향상:** 기본적인 구조가 이미 갖춰져 있으므로, 개발자는 복잡한 설정이나 반복적인 작업 대신 프로그램의 핵심 기능(코드 로직) 개발에만 집중할 수 있어요.
- **코드 로직 집중:** 예를 들어, **Django**라는 파이썬 프레임워크는 웹사이트의 회원 관리나 게시판 같은 기본적인 관리자 페이지를 미리 만들어줘요. 개발자는 이런 부가적인 기능에 시간을 낭비하지 않고, 웹사이트의 고유한 서비스 개발에만 힘쓸 수 있습니다.

IoC (Inversion of Control): 프레임워크의 가장 큰 특징 중 하나는 "**제어의 역전**"이라는 개념이에요. 보통은 개발자가 프로그램의 흐름을 직접 제어하지만, **프레임워크를 사용하면 프레임워크가 전체적인 흐름을 관리하고, 개발자는 프레임워크가 호출하는 특정 부분만 구현하게 돼요.** 예를 들어, 놀이공원 가이드가 전체 투어 코스를 정하고 각 코스에서 무엇을 할지만 여러분이 결정하는 것과 비슷해요.

언어별 다양성: 자바스크립트에는 **React, Vue, Angular** 같은 프레임워크가 있고, 파이썬에는 **Django, Flask**, 그리고 PHP에는 **Laravel, CodeIgniter** 등 다양한 언어별 프레임워크들이 존재해요. 각 프레임워크마다 제공하는 도구와 기능이 조금씩 다르답니다.

5. [수업자료] 검색엔진 최적화 SEO

01 SEO의 개념

SEO(검색엔진 최적화)는 우리 웹사이트나 쇼핑몰이 **검색 결과 상단에 잘 노출되도록 하는 일련의 작업**을 의미합니다. 사용자들이 주로 검색(네이버, 구글)이나 SNS 광고를 통해 쇼핑몰에 접속하므로, 검색 결과 상단 노출은 매우 중요합니다. 이를 위해 **키워드 설정**과 **검색엔진 알고리즘**에 대한 이해가 필요합니다.

쉬운설명

SEO는 왜 필요할까요? 간단히 말해, 당신이 온라인 쇼핑몰을 운영한다고 가정해보세요. 아무리 좋은 상품을 팔아도 사람들이 당신의 쇼핑몰을 찾지 못하면 소용이 없겠죠? SEO는 사람들이 네이버나 구글 같은 검색엔진에서 특정 상품을 검색했을 때, 당신의 쇼핑몰이 **검색 결과 첫 페이지, 그것도 상단에 나타나도록 돕는 기술**입니다. 마치 상점들이 유동인구가 많은 좋은 자리에 입점하려는 것과 같아요.

어떻게 하면 잘 보일 수 있을까요?

- **키워드 설정:** 사람들이 어떤 단어를 검색해서 당신의 쇼핑몰로 들어올지 예측하고, 그 단어들을 웹사이트 콘텐츠에 잘 녹여내는 것입니다. 예를 들어, '여성 쇼핑몰'이 아니라 '20대 여성 오피스룩 쇼핑몰'처럼 구체적인 키워드를 사용하는 것이죠.
- **알고리즘 이해:** 검색엔진은 복잡한 규칙(알고리즘)에 따라 웹사이트 순위를 결정합니다. 이 알고리즘이 "어떤 웹사이트가 사용자에게 유용할까?"를 판단하는 기준을 이해하고, 그 기준에 맞춰 웹사이트를 개선하는 것이 중요합니다. 예를 들어, 웹사이트 속도가 빠르거나, 정보가 최신이거나, 다른 좋은 사이트에서 링크를 많이 걸어주는 경우 등이죠.

결국 **SEO는 검색엔진과 사용자의 입맛에 맞춰 웹사이트를 잘 꾸미고 홍보하는 마케팅 전략**이라고 볼 수 있습니다!

02 현대 검색엔진의 시작

현대 검색엔진의 개념은 **1996년에 시작**되었으며, 웹사이트를 다운로드, DB 저장, 웹 구성요소 분류 및 분석하는 연구 프로세스를 통해 최적의 검색 알고리즘을 만들어 가는 방식으로 발전했습니다. 프로젝트 기간이 길어짐에 따라 서버를 증설하여 방대한 웹 데이터를 처리했으며, **구글(Google)**의 공동 창업자인 **래리 페이지(Larry Page)**와 **세르게이 브린(Sergey Brin)**이 이러한 현대 검색엔진의 시초가 되었습니다.

쉬운설명

현대 검색엔진은 어떻게 시작되었을까요? 아주 쉽게 말해서, 인터넷이 처음 생겼을 때는 웹사이트가 그렇게 많지 않았어요. 하지만 웹사이트가 점점 늘어나면서 사람들이 원하는 정보를 찾기가 어려워졌죠. 이때 "어떻게 하면 웹사이트들을 효과적으로 정리해서 사람들이 쉽게 찾을 수 있게 할까?"라는 고민에서 검색엔진이 탄생했습니다.

초기 검색엔진의 작동 방식:

1. **웹사이트 다운로드:** 검색엔진은 인터넷에 있는 모든 웹사이트를 찾아다니며 마치 책을 수집하듯이 정보를 다운로드합니다.
2. **DB 저장:** 다운로드한 정보들을 자신들의 커다란 창고(데이터베이스, DB)에 차곡차곡 저장해요.

3. **웹 구성요소 분류 및 분석:** 저장된 웹사이트의 내용을 분석해서 "이 웹사이트는 음식에 대한 정보를 담고 있군", "이 웹사이트는 쇼핑몰이네" 하는 식으로 분류하고 어떤 내용이 중요한지 분석합니다. 이 과정을 통해 사용자가 검색했을 때 가장 적합한 결과를 보여줄 수 있는 똑똑한 규칙(알고리즘)을 만드는 것이죠.

초기에는 웹사이트 수가 적었지만, 인터넷이 폭발적으로 성장하면서 다룰 데이터가 많아지자, 검색엔진 회사들은 더 많은 컴퓨터(서버)를 추가하고(서버 증설) 더 복잡하고 효율적인 알고리즘을 개발하기 시작했습니다. 그리고 이러한 노력의 중심에 바로 지금 우리가 사용하는 **구글(Google)** 이 있었던 것입니다. 구글의 창업자들은 웹사이트들의 중요도를 판단하는 혁신적인 방법을 제시하며 현대 검색엔진 시대를 열었어요.

03 검색엔진 최적화

검색엔진 최적화는 **검색엔진이 새로운 사이트를 탐색하고 분류하는 과정을 이해하고 활용하는 것**입니다. 전 세계적으로 114억 개에 달하는 웹사이트가 존재하며, 이 중에서 자신의 웹사이트를 효과적으로 노출시키기 위한 전략이 필요합니다.

핵심 키워드를 통한 노출 전략은 물론, SNS에서 사용되는 **해시태그(#)** 또한 검색 최적화 시도 중 하나입니다. 또한, **네이버 웹마스터 도구**나 **구글 서치 콘솔** 등을 통해 사이트를 직접 등록하여 검색엔진이 내 웹페이지를 더 쉽게 찾을 수 있도록 할 수 있습니다. 네이버는 **Search Advisor**를 통해 웹사이트 성장을 지원합니다.

구글의 검색엔진은 크게 3단계로 작동합니다:

1. **크롤링(Crawling):** 신규 또는 업데이트된 웹페이지를 찾는 과정입니다. Googlebot과 같은 크롤러가 웹을 탐색하여 페이지를 가져옵니다.
2. **색인 생성(Indexing):** 크롤링한 웹페이지의 콘텐츠 및 위치(URL)를 Google의 색인에 저장하는 과정입니다. 이는 검색 속도를 높이기 위해 데이터를 분류하고 저장하는 방법입니다.
3. **최종 검색결과 제공:** 색인된 정보를 바탕으로 사용자에게 최적의 검색 결과를 제공합니다.

구글의 검색엔진 최적화 문서에서 제시하는 용어:

용어	정의
색인	Google이 알고 있는 모든 웹페이지를 저장하는 것
크롤링	신규 또는 업데이트된 웹페이지를 찾아 색인에 추가하는 프로세스
크롤러	웹페이지를 크롤링하여 색인을 생성하는 자동 소프트웨어
Googlebot	Google 크롤러의 일반적인 이름
SEO	검색엔진에서 찾기 쉽도록 사이트를 개선하는 프로세스

쉬운설명

검색엔진 최적화, 제대로 파헤쳐볼까요? 검색엔진 최적화(SEO)는 단순히 웹사이트를 만드는 것을 넘어, 검색엔진이 "와, 이 사이트 정말 좋다!"라고 생각하게 만드는 과정이에요. 수십억 개의 웹사이트 중에서 내 사이트가 눈에 띄게 하려면 어떻게 해야 할까요?

핵심 전략:

- **키워드 파악 및 활용:** 사람들이 어떤 단어를 검색해서 내 사이트에 올지 먼저 생각해야 해요. 예를 들어, '운동화' 대신 '편한 러닝화 추천'처럼 구체적인 키워드를 웹사이트 제목, 내용, 설명 등에 잘 배치하는 거죠.

- **해시태그(#) 활용:** 인스타그램이나 페이스북 같은 SNS에서 #맛집, #데일리룩처럼 해시태그를 사용하는 것도 검색 최적화의 한 방법이에요. 검색엔진도 이러한 해시태그를 통해 콘텐츠의 주제를 파악하기도 하니까요.
- **사이트 직접 등록:** 검색엔진이 알아서 내 사이트를 찾아주길 기다리기만 할 수는 없겠죠? 네이버의 '웹마스터 도구'나 구글의 '서치 콘솔' 같은 곳에 직접 내 웹사이트 주소를 등록하면 검색엔진이 더 빠르고 정확하게 내 사이트를 인식하고 색인화할 수 있어요.

구글 검색엔진의 3단계 작동 방식:

1. **크롤링 (Crawling) - 정보 수집 단계:** 🕷 거미(크롤러)가 웹사이트를 기어 다니며 새로운 정보나 업데이트된 정보를 찾아다니는 과정이에요. 웹사이트 링크를 따라가거나 사이트맵(웹사이트 지도)을 읽으면서 "여기 이런 페이지가 있네!" 하고 발견하는 거죠. **Google에서는 이 거미를 Googlebot이라고 부릅니다.**
2. **색인 생성 (Indexing) - 정보 정리 단계:** 📁 수집된 정보를 단순히 모으는 것을 넘어, 마치 도서관의 책을 분류하고 정리하듯이 검색엔진이 알아보기 쉽도록 정리하고 저장하는 과정이에요. 예를 들어, "이 페이지는 '강아지 간식'에 대한 내용이고, 이 키워드로 검색하면 보여줘야겠다" 하고 목록에 올리는 거죠. **색인이 잘 되어 있어야 나중에 검색했을 때 빠르게 찾아낼 수 있어요.**
3. **최종 검색결과 제공 - 정보 제공 단계:** 🔍 사용자가 검색창에 키워드를 입력하면, 색인된 정보들 중에서 가장 관련성이 높고 유용한 페이지를 찾아 사용자에게 보여주는 단계입니다. 이 과정에서 검색엔진의 복잡한 알고리즘이 작동하여 순위를 결정하고 결과를 보여줍니다.

결국 **SEO는 이 세 단계를 검색엔진이 잘 수행하도록 우리의 웹사이트를 최적화하는 작업이라고 할 수 있어요.** ✨

04 크롤링

크롤링(Crawling)은 컴퓨터를 활용하여 **웹페이지의 정보를 자동으로 그대로 긁어와 가져오는 행위**를 말합니다. 웹에 있는 HTML 코드 형태를 그대로 가져오는 것이죠.

크롤링된 데이터에서 필요한 정보를 추출하는 과정을 **파싱(Parsing)**이라고 합니다. 파싱은 구문 분석을 의미하며, 특정 규칙에 따라 문장을 잘라내 분석하는 방법입니다. 웹사이트를 크롤링하여 얻은 정보 중에서 가격 정보와 같이 원하는 부분만 **** 블록 안에서 추출하는 것이 그 예입니다.

크롤링과 파싱 프로세스:

1. **웹사이트 크롤링:** 웹사이트의 전체 데이터를 수집합니다.
2. ** Parsing을 통한 가격정보 획득:** 수집된 데이터에서 필요한 가격 정보를 파싱하여 추출합니다.
3. **최종 결과를 DB에 저장 or 출력:** 추출된 정보를 데이터베이스에 저장하거나 사용자에게 출력합니다.

예를 들어, 포켓몬 빵에서 포장지와 빵을 분류하고 목표인 포켓몬 스티커만 획득하는 과정과 유사합니다.

모든 사이트를 크롤링할 수 있는 것은 아니며, 검색 사이트 노출 방지 방법도 존재합니다.

크롤링의 목적:

- 시장 가격 조사를 위한 데이터 모음 (예: 쇼핑몰 판매)
- AI 훈련을 위한 데이터 확보
- 가격 비교 사이트

크롤링의 이슈

- **사이트의 트래픽 발생:** 크롤링할 사이트에 접속하면 트래픽이 발생합니다.
- **유지비용 증대 및 속도 저하:** 크롤링으로 인한 트래픽 비용은 크롤링하는 사람이 아닌 웹사이트 관리자가 부담하며, 많은 사람이 크롤링을 하면 운영 비용이 증가하고 사이트 속도가 저하될 수 있습니다.

크롤링의 합법성 논란:

미국 항소법원은 공개적으로 접근 가능한 데이터의 웹 스크래핑(크롤링)이 합법이라고 재확인했습니다. 한국에서는 **정보통신망법 제48조** '정보통신망 침해행위 금지' 규정에 따라 '정당한 접근권한 없이 또는 허용된 접근권한을 넘어 정보통신망에 침입하여서는 아니 된다'고 명시되어 있어 합법과 불법의 경계가 모호할 수 있습니다. 예를 들어, '여기어때' 무죄 판결 사례처럼 접근 권한을 침해하지 않았다면 합법으로 판단될 수 있습니다.

데이터의 가치가 그 어느 때보다 크기 때문에 크롤링 관련 이슈가 지속적으로 발생합니다. 구글 트렌드와 네이버 데이터 랩처럼 검색어 동향이나 인기 검색어 현황을 제공하는 서비스는 데이터의 중요성을 보여줍니다.

색인(Indexing) 은 데이터를 분류하여 쉽게 표시하는 과정입니다. 포털 서비스들의 웹 주소(예: [site:naver.com](https://site.naver.com) 검색)를 통해 색인을 가장 쉽게 확인할 수 있습니다. 네이버의 다양한 서비스 검색이 가능한 것은 네이버가 크롤링 이후 색인을 통해 분류하는 엔진 기능을 갖추고 있기 때문입니다. 온라인 마케터는 색인 과정에서 더 효율적인 노출이 될 수 있도록 작업합니다.

쉬운설명

크롤링(Crawling)과 파싱(Parsing)을 쉽게 이해해 볼까요? 이 두 단어는 웹의 방대한 정보를 수집하고 가공하는 데 아주 중요한 역할을 합니다.

크롤링(Crawling): 웹에서 정보 '긁어오기' 

크롤링은 웹사이트에 있는 수많은 정보를 컴퓨터 프로그램이 자동으로 방문해서 그대로 복사해 오는 과정이에요. 마치 도서관의 모든 책을 읽고 내용을 전부 복사해 오는 것과 같습니다.

예를 들어, 다나와 같은 가격 비교 사이트가 여러 쇼핑몰의 상품 정보를 가져올 때 크롤링을 사용합니다. 특정 상품 페이지의 HTML 코드를 통째로 가져오는 거죠.

파싱(Parsing): 긁어온 정보에서 '우리가 정말 원하는 것만 골라내기' 

크롤링으로 가져온 정보는 방대하고 복잡합니다. 파싱은 이 복잡한 정보 덩어리에서 **필요한 데이터만 쏙쏙 골라내는** 과정이에요.

예를 들어, 다나와에서 상품 가격 정보를 가져왔을 때, 우리는 상품 이미지나 설명이 아니라 '가격' 숫자만 필요하잖아요? 이럴 때 파싱을 통해 `<strong>1,899,000</strong>원`이라는 HTML 코드에서 **'1,899,000'**이라는 숫자만 추출하는 거예요.

비유: 포켓몬 빵을 샀을 때, 빵과 포장지는 버리고 안에 들어있는 포켓몬 스티커만 쏙 빼내는 과정이 파싱과 비슷합니다.

크롤링과 파싱은 왜 할까요? (목적)

- **시장 조사:** 경쟁 쇼핑몰의 가격 동향을 자동으로 파악하거나, 특정 상품의 시장 가격을 조사할 때 유용해요.
- **AI 훈련:** 인공지능이 더 똑똑해지려면 많은 데이터가 필요해요. 웹에서 다양한 텍스트나 이미지를 크롤링해서 AI 학습 데이터로 활용할 수 있습니다.
- **가격 비교 사이트:** 여러 쇼핑몰의 같은 상품 가격을 비교해서 보여주는 사이트들은 크롤링과 파싱을 통해 실시간으로 가격 정보를 업데이트합니다.

크롤링의 그림자 (이슈)

- **서버 부담:** 크롤러가 너무 자주, 너무 많은 웹사이트를 방문하면 해당 웹사이트의 서버에 과부하가 걸릴 수 있어요. 마치 너무 많은 사람이 한꺼번에 가게에 몰려와 서비스가 느려지는 것과 같죠. **이 비용은 크롤링을 하는 사람이 아니라 웹사이트 운영자가 부담해야 합니다.**

- **법적 문제:** 크롤링 자체는 합법적인 경우가 많지만, 접근 권한이 없는 정보를 가져오거나, 상업적으로 무단 도용하는 경우에는 불법이 될 수 있어요. 예를 들어, '여기어때' 사례처럼 기업 간에 크롤링으로 인한 분쟁이 발생하기도 합니다. (결국은 '데이터'의 가치가 너무 커서 이런 문제가 생기는 것이죠.)

색인(Indexing): 데이터의 분류와 표시

크롤링과 파싱을 통해 얻은 수많은 데이터를 단순히 저장만 하는 것이 아니라, 검색엔진이 더 빠르고 정확하게 찾아낼 수 있도록 잘 분류하고 정리하는 것이 '색인'입니다.

우리가 `site:naver.com`이라고 구글에 검색하면, 구글이 알고 있는 네이버 관련 페이지 목록이 쭉 뜨죠? **이것이 바로 구글이 네이버 웹사이트를 크롤링하고 색인화했다는 증거입니다.**

온라인 마케터들은 이 색인 과정에서 자신들의 웹사이트가 더 잘 노출되도록 제목, 설명, 키워드 등을 최적화하는 작업을 합니다. ✨

6. 현업에서의 용어 노하우

Version Control

Version Control은 전체 프로젝트의 개발, 출시, 버그 수정 등 모든 단계에서 **코드를 효과적으로 관리하는 방법**입니다. 이를 통해 출시 및 개발 시 기능들을 체계적으로 관리하고, 문제가 발생했을 때 **이전 버전으로 쉽게 되돌릴 수 있습니다**. **Git**은 이 분야에서 사실상 표준으로 자리 잡았으며, 개발자들이 버전을 나누어 테스트, 출시, 제작 등의 작업을 편리하게 수행할 수 있도록 돕습니다. 또한 **누가 코드를 작성했는지 추적 가능하여 협업에 매우 유용합니다**. **GitHub**과 **Bitbucket**은 Git을 편리하게 사용할 수 있도록 서버 입장에서 지원하는 대표적인 서비스들입니다.

쉬운설명

버전 컨트롤(Version Control)은 쉽게 말해, 우리가 문서를 작성하거나 코딩을 할 때 "저장하기" 버튼을 계속 누르며 여러 개의 사본을 만드는 것과 비슷해요. 하지만 단순한 저장보다는 훨씬 똑똑하게 관리해줍니다.

왜 필요할까요?

- **실수 만회!** 🕒: 만약 최신 버전에 치명적인 버그가 생겼다면? 버전 컨트롤을 사용하면 "이전의 안정적인 버전"으로 단숨에 돌아갈 수 있어요. 마치 타임머신을 타고 과거로 돌아가 실수를 없었던 일로 만드는 것과 같아요.
- **협업의 필수!** 👥: 여러 사람이 함께 프로젝트를 진행할 때, 각자 작업한 내용을 합쳐야 하잖아요? 버전 컨트롤은 누가 어떤 부분을 수정했는지 **명확하게 기록하고, 충돌 없이 합칠 수 있도록 도와줍니다**. 누가 '설탕 1스푼'을 넣었고, 누가 '설탕 2스푼'으로 바꿨는지 투명하게 알 수 있는 거죠.
- **이력 관리!** 📄: 프로젝트가 어떻게 발전해왔는지, 어떤 기능이 추가되고 변경되었는지 모든 기록이 남습니다. 이는 나중에 문제가 생겼을 때 원인을 파악하거나, 새로운 개발자가 프로젝트에 참여할 때 흐름을 이해하는 데 큰 도움이 됩니다.

가장 유명한 도구는? Git!

Git은 현재 버전 컨트롤 시스템의 거의 모든 것을 통일했다고 할 수 있을 정도로 가장 많이 사용됩니다.

Git을 기반으로 **GitHub**과 **Bitbucket** 같은 서비스들이 있어요. 이 서비스들은 Git으로 관리되는 코드들을 인터넷에 저장하고 공유하며, 팀원들이 함께 작업하기 편리하게 해줍니다. 마치 개인 작업실(Git)을 온라인 작업 공간(GitHub/Bitbucket)으로 확장하는 것과 같아요.

결론적으로, 버전 컨트롤은 소프트웨어 개발 프로젝트의 안정성, 효율성, 그리고 협업 능력을 극대화하는 핵심적인 도구입니다.

기획과 플로우 차트

기획과 플로우 차트는 어떤 서비스를 만들 것인지, 그 서비스의 핵심적인 흐름을 데이터와 함께 **시각적으로 보여주는 것**을 의미합니다. 쇼핑몰의 경우, '상품 페이지 접속' → '상품 구매' → '상품 배송'과 같은 핵심 프로세스를 플로우 차트로 나타낼 수 있습니다. 만약 초기 기획에서 배송 정보 획득 과정이 누락되었다면, '로그인' 단계를 추가하여 필요한 정보를 자연스럽게 얻도록 플로우를 개선할 수 있습니다. 로그인 후 구매 과정에서는 간편결제, 카드결제, 무통장 입금 등 다양한 결제 수단을 포함하며, 배송 과정에서는 배송사 선정 및 배송 과정을 **API 레퍼런스 체크**를 통해 제공할 수 있습니다. 프로토타입 이후에는 기획팀의 상세 기획이 시작되고, 개발자는 관련 레퍼런스, 라이브러리, API 등을 점검하며 실제 개발을 준비합니다.

쉬운설명

기획과 플로우 차트는 마치 건물을 짓기 전에 "설계도"를 그리는 것과 같아요. 어떤 서비스를 만들지, 사용자가 어떤 순서로 서비스를 이용하게 될지 **한눈에 알아볼 수 있도록 그림으로 표현하는 거죠.**

왜 필요할까요?

- **명확한 이해!**  : 기획자와 개발자, 디자이너 등 모든 팀원이 서비스의 흐름을 똑같이 이해할 수 있도록 도와줍니다. "이 버튼을 누르면 다음엔 뭐가 나와요?" 같은 불필요한 질문을 줄여줘요.
- **문제점 발견!**  : 쇼핑몰 예시처럼, 처음에 '상품 페이지 접속 → 상품 구매 → 상품 배송'만 생각했다면 "어? 배송지는 언제 입력받지?"라는 **중요한 누락을 쉽게 발견할 수 있어요.** 플로우 차트를 그려보면 이런 빈틈이 한눈에 보입니다.
- **효율적인 개발!**  : 설계도가 잘 되어 있으면, 개발자는 어떤 기능을 먼저 만들고, 어떤 정보가 필요한지 명확히 알 수 있어 헤매지 않고 효율적으로 개발할 수 있습니다.

플로우 차트 그리는 방법:

1. 시작점과 끝점을 정해요.
2. 각 단계를 상자나 원 같은 도형으로 표현해요.
3. 화살표로 단계 간의 흐름을 연결해요. 예를 들어, '로그인' 단계에서 사용자의 '이름, 전화번호, 주소지' 정보를 획득하고, '구매' 단계에서 '간편결제, 카드결제' 등을 선택하도록 하는 식으로요.
4. 만약 흐름 중간에 의사결정이 필요하면 마름모 같은 도형을 사용해서 "예/아니오" 같은 분기를 표시할 수도 있습니다.

플로우 차트는 단순히 그림을 그리는 것을 넘어, **서비스의 논리적인 구조와 사용자 경험을 미리 설계하고 검토하는 중요한 과정**입니다.

오픈소스

오픈소스는 프로젝트의 전체 코드가 공개되어 **누구나 자유롭게 활용하고, 수정하고, 배포할 수 있는 코드 소스**를 의미합니다. 많은 사람들의 기여로 프로젝트가 유지되고 발전하며, 대부분 무료로 사용 가능하지만, 엄연히 **라이선스에 따라 관리되고 있습니다.** 예를 들어, 안드로이드 스마트폰 운영체제도 오픈소스 기반이며, 우리가 사용하는 애플리케이션들은 메뉴에 자신들이 사용한 오픈소스 소프트웨어들을 기재하여 라이선스 규칙을 준수하고 있음을 보여줍니다. **OLIS**와 같은 오픈소스SW 라이선스 종합정보시스템을 통해 라이선스 정보를 확인할 수 있습니다.

쉬운설명

오픈소스(Open Source) 는 말 그대로 "코드가 활짝 열려있는" 소프트웨어예요. 누구든지 그 소프트웨어의 **설계도(소스 코드)** 를 자유롭게 보고, 가져다 쓰고, 고치고, 심지어 재배포할 수 있습니다.

오픈소스의 특징과 장점:

- **무료 사용 & 자유로운 활용** 🏠: 대부분의 오픈소스는 공짜로 쓸 수 있어요. 회사에서 특정 기능을 개발할 때, 바둑부터 코드를 짜는 대신 이미 만들어진 오픈소스를 가져다 쓰면 개발 시간과 비용을 크게 절약할 수 있습니다.
- **전 세계 개발자들의 협업** 🌐: 오픈소스 프로젝트는 전 세계 수많은 개발자들이 자발적으로 참여해서 버그를 찾고, 새로운 기능을 추가하며 발전시킵니다. 마치 집단 지성으로 더 좋은 결과물을 만들어내는 것과 같아요.
- **투명성과 안정성** 🔍: 코드가 공개되어 있기 때문에 누구나 코드를 검토할 수 있어요. 이는 보안 취약점이나 버그를 더 빨리 발견하고 수정하는 데 도움이 됩니다.
- **다양한 예시**: 우리가 매일 쓰는 안드로이드 운영체제도 오픈소스의 대표적인 예시예요. 웹 브라우저인 Firefox, 데이터베이스 MySQL 등도 오픈소스입니다.

⚠️ **라이선스! 중요한 점은 '무료'라고 해서 '내 마음대로' 쓸 수 있는 것은 아니라는 거예요.** 각 오픈소스에는 **라이선스(사용 허락 조건)** 가 있어서, 이 라이선스가 정한 규칙을 지키면서 사용해야 합니다. 예를 들어, 어떤 오픈소스는 "이 코드를 사용해서 제품을 만들면, 당신이 만든 제품의 소스 코드도 공개해야 한다"는 조건을 가지고 있기도 합니다. 카카오톡 같은 앱도 사용된 오픈소스 목록과 라이선스를 명시하고 있습니다.

오픈소스는 현대 IT 산업의 근간을 이루는 중요한 개념이며, 혁신과 발전을 가속화하는 원동력이 되고 있습니다.

클라우드

클라우드는 실물 서버 없이 **인터넷을 통해 서버와 소프트웨어를 활용하는 서비스**입니다. 기존 서버는 용량 확장 및 운영의 유연성이 부족하고, 한번 구축하면 지속적인 유지보수 비용이 발생하며, 계산이 틀렸을 때 추가 비용 위험이 있습니다. 반면 클라우드는 유연하게 셋업 가능하고, **사용한 만큼만 비용을 지불하는 구조**입니다. 또한 365일 서버를 가동해야 하는 운영 부담을 줄여주며, 서버 증설 및 폐기를 클릭 몇 번만으로 손쉽게 관리할 수 있습니다. **AWS, Microsoft Azure, Google Cloud** 등이 대표적인 클라우드 서비스입니다.

쉬운설명

클라우드(Cloud) 는 한마디로 "**인터넷을 통해 컴퓨터 자원(서버, 저장 공간, 소프트웨어 등)을 빌려 쓰는 것**" 이라고 생각하면 쉬워요. ☁️ 예전에는 직접 비싼 컴퓨터 서버를 사서 회사에 두고 관리해야 했지만, 이제는 구름(Cloud) 너머 어딘가에 있는 거대한 컴퓨터들을 인터넷으로 연결해서 필요한 만큼만 빌려 쓰는 거죠.

클라우드가 왜 좋을까요?

유연하고 저렴하게! 💡

- **옛날 방식 (기존 서버)**: 사업 시작할 때 '손님이 100명 올 거야!' 예상하고 비싼 서버를 샀는데, 막상 10명만 오면 서버는 놀고 돈은 낭비되죠. 반대로 1,000명이 몰리면 서버가 다운되어 버려요. 증설하려면 또 비싼 장비를 사고 설치해야 해서 시간이 오래 걸려요.
- **클라우드 방식**: 손님이 100명 오면 100명용 서버를 빌리고, 1,000명이 오면 클릭 몇 번으로 1,000명용 서버로 쉽게 늘릴 수 있어요! 그리고 손님이 줄어들면 다시 줄일 수도 있죠. **사용한 만큼만 돈을 내니 비용 낭비가 없습니다.**

관리 걱정 끝! 🛠️: 서버를 직접 관리하려면 365일 24시간 정상적으로 돌아가는지 신경 써야 하고, 고장 나면 고쳐야 하고... 할 일이 많아요. 클라우드를 쓰면 이런 운영 부담을 클라우드 제공 업체가 대신 해줍니다.

글로벌 서비스도 쉽게! 🌐: 전 세계 곳곳에 클라우드 데이터센터가 있어서, 한국뿐 아니라 미국, 유럽 등 다양한 지역의 사용자들에게 빠르고 안정적인 서비스를 제공할 수 있어요.

대표적인 클라우드 서비스:

- **AWS (Amazon Web Services):** 아마존이 제공하는 클라우드 서비스. 가장 크고 다양한 기능을 제공해요.
- **Microsoft Azure:** 마이크로소프트가 제공하는 클라우드 서비스.
- **Google Cloud (GCP):** 구글이 제공하는 클라우드 서비스.

클라우드를 IT 인프라를 구축하고 관리하는 방식을 혁신하여, 기업들이 기술적인 복잡성보다는 **자신들의 핵심 비즈니스에 집중할 수 있도록 돕습니다.**

SaaS

SaaS는 Software as a Service의 약자로, 클라우드 기반의 소프트웨어 서비스입니다. 사용자가 소프트웨어를 직접 설치할 필요 없이 인터넷을 통해 바로 사용할 수 있는 형태이며, 파일 공유와 같은 유연한 공유 기능을 제공합니다. 어도비(Adobe)의 피그마(Figma) **28조원 인수** 사례처럼, SaaS는 그 위력과 가능성이 계속해서 확장되고 있는 주요 서비스 모델입니다.

쉬운설명

SaaS (Software as a Service) 는 "서비스로서의 소프트웨어"를 의미해요. 우리가 흔히 쓰는 네이버 메일, 구글 문서, 드롭박스, 넷플릭스 등이 모두 SaaS의 대표적인 예시입니다.

SaaS의 핵심은 이것!

- **설치 NO! 접속 YES!** 🖥️: 예전에는 워드 프로그램을 사용하려면 CD를 사거나 파일을 다운로드해서 컴퓨터에 설치해야 했죠? 하지만 SaaS는 **별도의 설치 과정 없이, 웹 브라우저나 앱을 통해 바로 접속해서 사용할 수 있어요.** 마치 유튜브를 보려고 프로그램을 설치하지 않고 웹사이트에 접속하는 것과 같아요.
- **클라우드 기반** ☁️: SaaS는 클라우드 기술을 기반으로 작동해요. 그래서 어느 기기에서든 인터넷만 되면 내 계정으로 로그인해서 똑같은 환경에서 작업할 수 있습니다. 예를 들어, 구글 문서에 작성한 파일은 사무실 컴퓨터에서 작업하다가 집에서 노트북으로 이어서 작업할 수 있죠.
- **구독 모델** 📅: 대부분의 SaaS는 매달 또는 매년 사용료를 내는 구독(Subscription) 방식으로 운영됩니다. 초기 구매 비용 부담이 적고, 항상 최신 버전의 소프트웨어를 사용할 수 있다는 장점이 있어요.
- **협업 용이** 🤝: 여러 사람이 동시에 같은 문서를 편집하거나 파일을 공유하는 등의 **협업 기능이 매우 강력합니다.** 구글 문서, 슬랙 등이 대표적이죠.

SaaS의 성장:

어도비가 디자인 협업 툴인 피그마(Figma)를 **28조 원에 인수**한 사례는 SaaS 모델이 얼마나 큰 가치와 가능성을 가지고 있는지 보여줍니다. **소프트웨어를 제품으로 팔던 시대에서, 소프트웨어를 서비스로 제공하는 시대로 바뀌고 있는 것이죠.**

SaaS는 사용자에게 편리함과 유연성을 제공하고, 기업에게는 효율적인 서비스 제공 모델을 제시하며 빠르게 성장하고 있습니다.